Project Title

The Open Source Audit

Open Source Software
(Linux Administration)
(CSE0002)

Name : Adhikari Viswa Teja
Reg N.o : 24MIM10138

Semester : Winter Semester 2025-26
Software Choosen : Linux Kernel

Part A — Origin & Philosophy

A1. The problem Linux Kernel was created to solve

The Linux kernel was created mainly because existing systems at that time were either too expensive or too restricted. Back in the early 1990s, operating systems like UNIX were not easily available for students, and even if they were, you couldn't freely modify them. This made it hard for learners who actually wanted to understand how systems work internally.

From what I understood, Linus Torvalds was just a student who wanted a system he could experiment with. He was using MINIX, but it had limitations and wasn't fully open for modification. Instead of just accepting that, he decided to build his own kernel as a personal project.

The interesting part is that he didn't keep it private. He shared it online in 1991 and asked others to contribute. This turned a small personal project into something much bigger, because developers from all over the world started improving it together.

Basically, the Linux kernel solved the problem of accessibility and control. It gave people a system they could actually study, modify, and use without restrictions. Over time, it became the base for many systems we use today, like Android and most servers.

In my opinion, this shows that the main goal was not just to build software, but to create something open where people can learn and collaborate. That's what makes Linux different from many other systems.

A2. License

The Linux kernel uses the GPL v2 license, which is one of the main reasons it stayed open and widely used. From what I understand, this license is not just about making software free to use, but also about making sure it stays open for everyone in the future.

There are four main freedoms in open-source software. First, anyone can run the program for any purpose. Second, they can study how it works and modify it if needed. Third, they can share it with others. And fourth, they can even share their modified versions. The Linux kernel supports all these freedoms, which is why it is considered true free software.

One interesting thing about GPL is that it forces openness. If a company modifies the Linux kernel and distributes it, they have to release their changes as well. They can't just take it, improve it, and keep it private. This is very different from licenses like MIT, where companies can use the code and not share anything back.

Another thing I found interesting is the idea of "free as in freedom" vs "free as in free beer." The Linux kernel is more about freedom. It's not just free to use, but also gives control to the user. For example, even if a company uses Linux in their product, the source code is still available to the public.

In my opinion, GPL is a bit strict but in a good way. It protects the community and makes sure no one can take advantage of open-source work without giving something back.

A3. Ethics

Open source is not just about code, it's more about how knowledge is shared. From what I understand, it follows the idea that software should be open so anyone can learn from it and improve it. But at the same time, not everything is that simple.

One question is whether all software should be open source. On one side, I think making everything open source would help in learning and innovation. More people can access tools and improve them, which can lead to faster development. But on the other side, companies invest a lot of time and money into building software. If everything is open, they might not get enough return for their effort. So I feel both open source and proprietary software have their place.

Another interesting point is whether it is ethical for companies to make profit using open-source software. Many companies use the Linux kernel in their products and still earn money from it. At first, it feels a bit unfair because the base work is done by the community. But at the same time, these companies also contribute back in different ways, like improving the code or maintaining it. So I think it depends on whether they are giving back to the community or just using it.

The idea of "standing on the shoulders of giants" is very relevant here. It means that new innovations are built on top of existing work. Open source makes this easier because everything is available to study and use. In my opinion, this does not reduce originality, but actually increases it, because people can focus on improving things instead of starting from zero.

Overall, I think open source is a very powerful concept. It promotes sharing and collaboration, but it also needs balance so that both individuals and companies benefit from it.

Part B — Linux Footprint

B1. Installation Method

The Linux kernel is not installed like normal software because it is already a core part of the operating system. In my case, since I am using Fedora, the kernel comes pre-installed with the OS. However, it can still be updated using the system package manager.

In Fedora, the kernel is managed using the dnf package manager. Updates are provided regularly by the community and can be installed using simple commands like sudo dnf update. This makes it easy to keep the system secure and up to date without manually compiling the kernel.

```
shiro@shiros-fedora:~$ sudo dnf update
[sudo] password for shiro:
Updating and loading repositories:
 Visual Studio Code                                                    100% |   1.3 KiB/s |   1.5 KiB |  00m01s
 RPM Fusion for Fedora 43 - Nonfree - Updates                          100% | 646.0   B/s |  14.7 KiB |  00m23s
 RPM Fusion for Fedora 43 - Nonfree - Steam                            100% | 696.0   B/s |  14.8 KiB |  00m22s
 RPM Fusion for Fedora 43 - Nonfree - NVIDIA Driver                    100% | 923.0   B/s |  15.5 KiB |  00m17s
 RPM Fusion for Fedora 43 - Free - Updates                             100% |   3.0 KiB/s |  11.6 KiB |  00m04s
 Copr repo for PyCharm owned by phracek                                100% |   1.8 KiB/s |   2.1 KiB |  00m01s
 google-chrome                                                         100% |   1.1 KiB/s |   1.3 KiB |  00m01s
 Fedora 43 - x86_64 - Updates                                          100% |   2.9 KiB/s |   8.6 KiB |  00m03s
Repositories loaded.
Package                                    Arch     Version                          Repository                    Size
Removing:
 kernel                                    x86_64   6.18.10-200.fc43                 <unknown>                  0.0   B
 kernel-core                               x86_64   6.18.10-200.fc43                 <unknown>                 97.6 MiB
 kernel-modules                            x86_64   6.18.10-200.fc43                 <unknown>                 96.3 MiB
 kernel-modules-core                       x86_64   6.18.10-200.fc43                 <unknown>                 68.0 MiB
 kernel-modules-extra                      x86_64   6.18.10-200.fc43                 <unknown>                  4.2 MiB
```

B2. Key Directories

The Linux kernel itself does not exist as a single file like a normal program. Instead, it is spread across different directories in the system. The kernel image is usually found in the /boot directory, where files like vmlinuz are stored.

Configuration files related to the system are present in /etc, while logs that help in debugging and monitoring are stored in /var/log. System binaries and essential commands are located in directories like /usr/bin and /bin.

From my observation, these directories are organized in a way that separates system-level files from user-level files, which helps in better management and security.

```
shiro@shiros-fedora:~$ ls /boot
config-6.18.10-200.fc43.x86_64                      initramfs-6.18.13-200.fc43.x86_64.img  System.map-6.18.10-200.fc43.x86_64
config-6.18.13-200.fc43.x86_64                      initramfs-6.19.7-200.fc43.x86_64.img   System.map-6.18.13-200.fc43.x86_64
config-6.19.7-200.fc43.x86_64                       loader                                 System.map-6.19.7-200.fc43.x86_64
efi                                                 lost+found                             vmlinuz-0-rescue-2daab5b5e6d94da580317b2d09879d9f
grub2                                               symvers-6.18.10-200.fc43.x86_64.xz     vmlinuz-6.18.10-200.fc43.x86_64
initramfs-0-rescue-2daab5b5e6d94da580317b2d09879d9f.img symvers-6.18.13-200.fc43.x86_64.xz vmlinuz-6.18.13-200.fc43.x86_64
initramfs-6.18.10-200.fc43.x86_64.img               symvers-6.19.7-200.fc43.x86_64.xz      vmlinuz-6.19.7-200.fc43.x86_64
```

B3. User and Group

The Linux kernel itself does not run as a normal user process. It operates at a very low level and directly interacts with the hardware. However, most system services that depend on the kernel run under specific users and groups for security reasons.

For example, in Linux systems, services often run under restricted users instead of the root user to prevent misuse. This is important because if everything ran as root, any small issue could affect the entire system.

```
shiro@shiros-fedora:~$ whoami
shiro
shiro@shiros-fedora:~$ uname -r
6.19.7-200.fc43.x86_64
```

B4. Service Management

The Linux kernel is loaded at the time of system boot and is not started or stopped like a normal service. However, the system uses tools like systemctl to manage services that depend on the kernel.

For example, commands like systemctl status, start, and stop are used to control system services. Although the kernel itself is always running, these services interact with it to perform various tasks.

B5. Update Model

The Linux kernel is continuously updated by developers around the world. These updates include security fixes, performance improvements, and support for new hardware.

In Fedora, updates are delivered through the official repositories and can be installed using the dnf package manager. This shows how the open-source community works together to maintain and improve the software over time.

Part C — FOSS Ecosystem

The Linux kernel does not work alone. It is part of a large open-source ecosystem where many tools and technologies depend on it. The kernel acts as the core layer that connects hardware and software, so almost every application in a Linux system depends on it indirectly.

One important dependency is the GNU tools, such as the compiler and basic system utilities. These tools work together with the Linux kernel to create a complete operating system. Without these supporting tools, the kernel alone would not be enough for users to interact with the system.

The Linux kernel has also enabled the development of many other technologies. For example, Android is built on top of the Linux kernel, and most servers on the internet use Linux-based systems. This shows how one open-source project can influence many others.

In terms of the web ecosystem, the Linux kernel plays a key role in the LAMP stack (Linux, Apache, MySQL, PHP). It provides the base on which web servers run. Without Linux, many modern websites and applications would not function the way they do today.

Another important part of the ecosystem is the community. The Linux kernel is developed and maintained by developers from all around the world. There are mailing lists, forums, and conferences where contributors discuss improvements and fix issues. This collaborative model is one of the biggest strengths of open source.
In my opinion, the Linux ecosystem is a great example of how different open-source projects can work together. Instead of competing, they support each other and create a strong and reliable system.

Part D — Open Source vs Proprietary

Dimension	Open Source (Linux Kernel)	Proprietary Alternative (Windows Kernel)
Cost	Free to use and distribute	Requires paid license
Security	Source code is open, so anyone can inspect and fix issues	Code is closed, only company can fix issues
Support & Reliability	Community support + some enterprise support (like Red Hat)	Official company support (Microsoft)
Freedom to Modify	Users can modify and customize the kernel	No permission to modify source code
Control	Community-driven development	Controlled by a single company

Conclusion

After comparing both, I feel that open-source software like the Linux kernel gives more freedom and flexibility to users. It allows developers to understand and modify the system according to their needs, which is very useful for learning and innovation.

However, proprietary software like Windows also has its advantages. It provides dedicated support and is often easier for general users who do not want to deal with technical details. So both have their own place depending on the use case.

In my opinion, for servers, development, and learning purposes, the Linux kernel is a better choice because of its openness and strong community support. I would definitely recommend it for real-world deployment in such scenarios.

If given a chance, I would also like to contribute to open-source projects in the future. Even small contributions can help improve the system and support the communite.

Shell Scripts Task

Script 1 — System Identity Report

Code:

```
GNU nano 8.5                                                    script1.sh
#!/bin/bash
# Script 1: System Identity Report
# Author:Adhikari Viswateja

STUDENT_NAME="Adhikari Viswateja"
SOFTWARE="Linux Kernel"

KERNEL=$(uname -r)
KERNEL_NAME=$(uname -s)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
DISTRO=$(grep PRETTY_NAME /etc/os-release | cut -d '"' -f2)

echo "======================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "======================================"
echo "Software        : $SOFTWARE"
echo "OS              : $DISTRO"
echo "Kernel Name     : $KERNEL_NAME"
echo "Kernel Version  : $KERNEL"
echo "User            : $USER_NAME"
echo "Home Directory  : $HOME"
echo "Uptime          : $UPTIME"
echo "Date & Time     : $DATE"
echo ""
echo "Linux Kernel is licensed under GPL v2"
echo "======================================"
```

Output:

```
+
shiro@shiros-fedora:~/oss_project$ chmod +x script1.sh
shiro@shiros-fedora:~/oss_project$ ./script1.sh
======================================
 Open Source Audit — Adhikari Viswateja
======================================
Software        : Linux Kernel
OS              : Fedora Linux 43 (Workstation Edition)
Kernel Name     : Linux
Kernel Version  : 6.19.7-200.fc43.x86_64
User            : shiro
Home Directory  : /home/shiro
Uptime          : up 2 hours, 56 minutes
Date & Time     : Sat Mar 28 02:59:52 PM IST 2026

Linux Kernel is licensed under GPL v2
======================================
```

Explanation:

This script displays basic system information such as kernel version, operating system, user name, uptime, and current date. It uses commands like uname, whoami, and uptime to fetch real-time system details. Overall, it works like a small system summary using simple shell scripting concepts.

Script 2 — Kernel Inspector

Code:

```
  GNU nano 8.5
#!/bin/bash
# Script 2: Kernel Inspector

PACKAGE="kernel"

# Check if kernel package exists
if rpm -q $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed."
    rpm -qi $PACKAGE | grep -E 'Version|License|Summary'
else
    echo "$PACKAGE is NOT installed."
fi

# Case statement
case $PACKAGE in
    kernel) echo "Linux Kernel: core of the operating system" ;;
    httpd) echo "Apache: powers web servers" ;;
    mysql) echo "MySQL: database system" ;;
    python) echo "Python: general purpose language" ;;
    *) echo "Unknown package" ;;
esac
```

Output:

```
shiro@shiros-fedora:~/oss_project
~/oss_project

shiro@shiros-fedora:~/oss_project$ chmod +x script2.sh
shiro@shiros-fedora:~/oss_project$ ./script2.sh
kernel is installed.
Version     : 6.18.10
License     : ((GPL-2.0-only WITH Linux-syscall-note) OR BSD-2-Clause) AND ((GPL-2.0-only WITH Linux-syscall-note) OR BSD-3-Clause) AND ((GPL-2.0-only WITH Linux-syscall-note) OR CDDL-1.0) AND ((GPL-2.0-only WIT
H Linux-syscall-note) OR Linux-OpenIB) AND ((GPL-2.0-only WITH Linux-syscall-note) OR MIT) AND ((GPL-2.0-or-later WITH Linux-syscall-note) OR BSD-3-Clause) AND ((GPL-2.0-or-later WITH Linux-syscall-note) OR MIT)
 AND BBSD AND BSD-2-Clause AND (BSD-2-Clause OR Apache-2.0) AND BSD-3-Clause AND BSD-3-Clause-Clear AND CC0-1.0 AND GFDL-1.1-no-invariants-or-later AND GPL-1.0-or-later AND (GPL-1.0-or-later OR BSD-3-Clause) AND
 (GPL-1.0-or-later WITH Linux-syscall-note) AND GPL-2.0-only AND (GPL-2.0-only OR Apache-2.0) AND (GPL-2.0-only OR BSD-2-Clause) AND (GPL-2.0-only OR BSD-3-Clause) AND (GPL-2.0-only OR CDDL-1.0) AND (GPL-2.0-onl
y OR GFDL-1.1-no-invariants-or-later) AND (GPL-2.0-only OR GFDL-1.2-no-invariants-only) AND (GPL-2.0-only OR GFDL-1.2-no-invariants-or-later) AND (GPL-2.0-only WITH Linux-syscall-note) AND GPL-2.0-or-later AND (
GPL-2.0-or-later OR BSD-2-Clause) AND (GPL-2.0-or-later OR BSD-3-Clause) AND (GPL-2.0-or-later OR CC-BY-4.0) AND (GPL-2.0-or-later WITH GCC-exception-2.0) AND (GPL-2.0-or-later WITH Linux-syscall-note) AND ISC A
ND LGPL-2.0-or-later AND (LGPL-2.0-or-later OR BSD-2-Clause) AND (LGPL-2.0-or-later WITH Linux-syscall-note) AND LGPL-2.1-only AND (LGPL-2.1-only OR BSD-2-Clause) AND (LGPL-2.1-only WITH Linux-syscall-note) AND
LGPL-2.1-or-later AND (LGPL-2.1-or-later WITH Linux-syscall-note) AND (Linux-OpenIB OR GPL-2.0-only) AND (Linux-OpenIB OR GPL-2.0-only OR BSD-2-Clause) AND Linux-man-pages-copyleft AND MIT AND (MIT OR Apache-2.0
) AND (MIT OR GPL-2.0-only) AND (MIT OR GPL-2.0-or-later) AND (MIT OR LGPL-2.1-only) AND (MPL-1.1 OR GPL-2.0-only) AND (X11 OR GPL-2.0-only) AND (X11 OR GPL-2.0-or-later) AND Zlib AND (copyleft-next-0.3.1 OR GPL
-2.0-or-later)
Summary     : The Linux kernel
Version     : 6.18.13
License     : ((GPL-2.0-only WITH Linux-syscall-note) OR BSD-2-Clause) AND ((GPL-2.0-only WITH Linux-syscall-note) OR BSD-3-Clause) AND ((GPL-2.0-only WITH Linux-syscall-note) OR CDDL-1.0) AND ((GPL-2.0-only WIT
H Linux-syscall-note) OR Linux-OpenIB) AND ((GPL-2.0-only WITH Linux-syscall-note) OR MIT) AND ((GPL-2.0-or-later WITH Linux-syscall-note) OR BSD-3-Clause) AND ((GPL-2.0-or-later WITH Linux-syscall-note) OR MIT)
 AND BBSD AND BSD-2-Clause AND (BSD-2-Clause OR Apache-2.0) AND BSD-3-Clause AND BSD-3-Clause-Clear AND CC0-1.0 AND GFDL-1.1-no-invariants-or-later AND GPL-1.0-or-later AND (GPL-1.0-or-later OR BSD-3-Clause) AND
 (GPL-1.0-or-later WITH Linux-syscall-note) AND GPL-2.0-only AND (GPL-2.0-only OR Apache-2.0) AND (GPL-2.0-only OR BSD-2-Clause) AND (GPL-2.0-only OR BSD-3-Clause) AND (GPL-2.0-only OR CDDL-1.0) AND (GPL-2.0-onl
y OR GFDL-1.1-no-invariants-or-later) AND (GPL-2.0-only OR GFDL-1.2-no-invariants-only) AND (GPL-2.0-only OR GFDL-1.2-no-invariants-or-later) AND (GPL-2.0-only WITH Linux-syscall-note) AND GPL-2.0-or-later AND (
GPL-2.0-or-later OR BSD-2-Clause) AND (GPL-2.0-or-later OR BSD-3-Clause) AND (GPL-2.0-or-later OR CC-BY-4.0) AND (GPL-2.0-or-later WITH GCC-exception-2.0) AND (GPL-2.0-or-later WITH Linux-syscall-note) AND ISC A
ND LGPL-2.0-or-later AND (LGPL-2.0-or-later OR BSD-2-Clause) AND (LGPL-2.0-or-later WITH Linux-syscall-note) AND LGPL-2.1-only AND (LGPL-2.1-only OR BSD-2-Clause) AND (LGPL-2.1-only WITH Linux-syscall-note) AND
LGPL-2.1-or-later AND (LGPL-2.1-or-later WITH Linux-syscall-note) AND (Linux-OpenIB OR GPL-2.0-only) AND (Linux-OpenIB OR GPL-2.0-only OR BSD-2-Clause) AND Linux-man-pages-copyleft AND MIT AND (MIT OR Apache-2.0
) AND (MIT OR GPL-2.0-only) AND (MIT OR GPL-2.0-or-later) AND (MIT OR LGPL-2.1-only) AND (MPL-1.1 OR GPL-2.0-only) AND (X11 OR GPL-2.0-only) AND (X11 OR GPL-2.0-or-later) AND Zlib AND (copyleft-next-0.3.1 OR GPL
-2.0-or-later)
Summary     : The Linux kernel
Version     : 6.19.7
License     : ((GPL-2.0-only WITH Linux-syscall-note) OR BSD-2-Clause) AND ((GPL-2.0-only WITH Linux-syscall-note) OR BSD-3-Clause) AND ((GPL-2.0-only WITH Linux-syscall-note) OR CDDL-1.0) AND ((GPL-2.0-only WIT
H Linux-syscall-note) OR Linux-OpenIB) AND ((GPL-2.0-only WITH Linux-syscall-note) OR MIT) AND ((GPL-2.0-or-later WITH Linux-syscall-note) OR BSD-3-Clause) AND ((GPL-2.0-or-later WITH Linux-syscall-note) OR MIT)
 AND BBSD AND BSD-2-Clause AND (BSD-2-Clause OR Apache-2.0) AND BSD-3-Clause AND BSD-3-Clause-Clear AND CC0-1.0 AND GFDL-1.1-no-invariants-or-later AND GPL-1.0-or-later AND (GPL-1.0-or-later OR BSD-3-Clause) AND
 (GPL-1.0-or-later WITH Linux-syscall-note) AND GPL-2.0-only AND (GPL-2.0-only OR Apache-2.0) AND (GPL-2.0-only OR BSD-2-Clause) AND (GPL-2.0-only OR BSD-3-Clause) AND (GPL-2.0-only OR CDDL-1.0) AND (GPL-2.0-onl
y OR GFDL-1.1-no-invariants-or-later) AND (GPL-2.0-only OR GFDL-1.2-no-invariants-only) AND (GPL-2.0-only OR GFDL-1.2-no-invariants-or-later) AND (GPL-2.0-only WITH Linux-syscall-note) AND GPL-2.0-or-later AND (
GPL-2.0-or-later OR BSD-2-Clause) AND (GPL-2.0-or-later OR BSD-3-Clause) AND (GPL-2.0-or-later OR CC-BY-4.0) AND (GPL-2.0-or-later WITH GCC-exception-2.0) AND (GPL-2.0-or-later WITH Linux-syscall-note) AND ISC A
ND LGPL-2.0-or-later AND (LGPL-2.0-or-later OR BSD-2-Clause) AND (LGPL-2.0-or-later WITH Linux-syscall-note) AND LGPL-2.1-only AND (LGPL-2.1-only OR BSD-2-Clause) AND (LGPL-2.1-only WITH Linux-syscall-note) AND
LGPL-2.1-or-later AND (LGPL-2.1-or-later WITH Linux-syscall-note) AND (Linux-OpenIB OR GPL-2.0-only) AND (Linux-OpenIB OR GPL-2.0-only OR BSD-2-Clause) AND Linux-man-pages-copyleft AND MIT AND (MIT OR Apache-2.0
) AND (MIT OR GPL-2.0-only) AND (MIT OR GPL-2.0-or-later) AND (MIT OR LGPL-2.1-only) AND (MPL-1.1 OR GPL-2.0-only) AND (X11 OR GPL-2.0-only) AND (X11 OR GPL-2.0-or-later) AND Zlib AND (copyleft-next-0.3.1 OR GPL
-2.0-or-later)
Summary     : The Linux kernel
Linux Kernel: core of the operating system
```

Explanation:

This script checks whether the kernel package is installed on the system. It uses rpm commands to display details like version, license, and summary. It also uses a case statement to print a short description based on the package name.

Script 3 — Disk and Permission Auditor

Code:

```
  GNU nano 8.5
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist"
    fi
done

# Kernel config check
if [ -d "/boot" ]; then
    ls -ld /boot
fi
```

Output:

```
+
shiro@shiros-fedora:~/oss_project$ chmod +x script3.sh
shiro@shiros-fedora:~/oss_project$ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x. root root | Size: 37M
/var/log => Permissions: drwxr-xr-x. root root | Size: 803M
/home => Permissions: drwxr-xr-x. root root | Size: 25G
/usr/bin => Permissions: dr-xr-xr-x. root root | Size: 514M
/tmp => Permissions: drwxrwxrwt. root root | Size: 8.0K
dr-xr-xr-x. 6 root root 4096 Mar 17 09:09 /boot
```

Explanation:

This script loops through important system directories and shows their size and permissions. It uses a for loop along with commands like ls and du to extract the required information. This helps in understanding how storage and access permissions are managed in the system.

Script 4 — Log File Analyzer

Code:

```
  GNU nano 8.5
#!/bin/bash
# Script 4: Log File Analyzer

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "File not found"
    exit 1
fi

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

# Show last matches
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Output:

```
shiro@shiros-fedora:~/oss_project
~/oss_project
shiro@shiros-fedora:~/oss_project$ chmod +x script4.sh
shiro@shiros-fedora:~/oss_project$ sudo ./script4.sh /var/log/messages
[sudo] password for shiro:
Keyword 'error' found 2159 times in /var/log/messages
Mar 28 14:52:14 shiros-fedora code.desktop[24824]: [24824:0328/145214.086106:ERROR:content/browser/browser_main_loop.cc:288] GLib-GObject: ../gobject/gsignal.c:2700: instan
ce '0x280c03aa7c00' has no handler with id '3270'
Mar 28 14:52:20 shiros-fedora code.desktop[24824]: [24824:0328/145220.763255:ERROR:content/browser/browser_main_loop.cc:288] GLib-GObject: ../gobject/gsignal.c:2700: instan
ce '0x280c03aa7240' has no handler with id '3449'
Mar 28 14:52:26 shiros-fedora code.desktop[24824]: [24824:0328/145226.597137:ERROR:content/browser/browser_main_loop.cc:288] GLib-GObject: ../gobject/gsignal.c:2700: instan
ce '0x280c03aa7c00' has no handler with id '3626'
Mar 28 14:52:44 shiros-fedora code.desktop[24824]: [24824:0328/145244.265453:ERROR:ui/ozone/platform/wayland/host/wayland_data_drag_controller.cc:165] Invalid state when tr
ying to start drag. source=kMouse
Mar 28 14:53:43 shiros-fedora code.desktop[24824]: [24824:0328/145343.794761:ERROR:content/browser/browser_main_loop.cc:288] GLib-GObject: ../gobject/gsignal.c:2700: instan
ce '0x280c03aa2440' has no handler with id '3801'
```

Explanation:

This script reads a log file and counts how many times a specific keyword appears. It uses a while loop and conditional checks to process each line of the file. It also displays the last few matching lines, which helps in identifying recent errors quickly.

Script 5 — Manifest Generator

Code:

```
  GNU nano 8.5
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "Answer 3 questions"

read -p "Tool you use: " TOOL
read -p "Meaning of freedom: " FREEDOM
read -p "What you want to build: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "On $DATE, I believe open source means $FREEDOM." > $OUTPUT
echo "Tools like $TOOL help people learn and grow." >> $OUTPUT
echo "In future, I want to build $BUILD and share it freely." >> $OUTPUT

echo "Saved to $OUTPUT"
cat $OUTPUT
```

Output:

```
+                                                    shiro@shiros-fedora:~/oss_project
                                                                          ~/oss_project
shiro@shiros-fedora:~/oss_project$ chmod +x script5.sh
shiro@shiros-fedora:~/oss_project$ ./script5.sh
Answer 3 questions
Tool you use: Linux
Meaning of freedom: freedom to modify
What you want to build: a useful open source project
Saved to manifesto_shiro.txt
On 28 March 2026, I believe open source means freedom to modify.
Tools like Linux help people learn and grow.
In future, I want to build a useful open source project and share it freely.
```

Explanation:

This script takes input from the user and generates a simple open-source manifesto. It uses the read command to collect user input and stores the result in a text file. This demonstrates basic input handling and file writing in shell scripting.